Workflow: SwayLayoutManager

Laboratorio de Investigación y Desarrollo de Software Libre (LIDSoL)

Areas:

Frontend
Backend

Date: December 17, 2025

Workflow Phases

1 Phase 1: Initial research and Planning

This phase will be a phase to focus on research for tools that can help for the development of the SwayLayoutManager project, also we will be planning the design and architecture of the project, by this I mean the actual code implementation structure, the modules that will compose the project, and how they will interact with each other.

1.1 Objectives

- Research existing tools and libraries that can facilitate interaction with Sway and Wayland.
- Propose a design and architecture for the SwayLayoutManager project.
- Create a plan with expected deadlines for the next phases of the project.

This phase won't last long since the heavy part for the command implementation is already defined in the proposal document. The main goal is to see if there is a more acurate way to implement commands reather that interact with `swaymsg` so we can can improve performance and reliability.

As mentioned in the proposal document, the programming language selected for the project CLI is Go, this can be changed if we find a better alternative during this research phase.

2 Phase 2: Development for parsing tools

This phase will focus on the actual development of the SwayLayoutManager tool, specifically on the parsing of the JSON presets that will be used to save layouts. This phase will be the first code implementation phase, where we will start writing the code that will form the basis of the project, taking into consideration the programming language and libraries selected in phase 1.

2.1 Save Command Implementation

The `swaylayoutmgr save <name>` command is the core functionality that captures the current state of all active workspaces and saves it as a named layout preset. This command will be implemented with the following detailed specification:

2.1.1 Command Execution Flow

1. **Workspace Discovery:** Execute `swaymsg -t get_tree | jq`(to be determined) to obtain the current layout tree and identify all active workspaces along with their contained windows.
2. **Data Parsing:** Parse the JSON output to extract relevant information for each window in the specified workspaces, including:
    - Application identifier (`app_id` for Wayland apps, `class` for X11 apps)
    - Window title (`name` field) for additional matching criteria
    - Workspace number/name (`workspace`)

- Container layout type (`layout`: `splith`, `splitv`, `tabbed`, `stacked`)
- Relative position within parent container (`rect` coordinates and sibling order)
- Workspace representation
- Window dimensions and floating status (`floating`, `rect`)
- Output/monitor association

3. **Preset Storage:** Save the structured data as a JSON file in `~/.config/swaylayoutmanager/presets/<` with metadata including creation timestamp and sway version compatibility.
4. **Validation:** Verify the preset file is valid JSON and contains all necessary fields before confirming successful save operation with log output.

2.1.2 Error Handling Requirements

- Check if preset name already exists and prompt for overwrite confirmation
- Validate that sway IPC socket is accessible (to be determined)
- Handle cases where some applications don't have reliable identifiers (to be determined)
- Create configuration directory structure if it doesn't exist
- Provide meaningful error messages for troubleshooting

2.1.3 JSON Structure Design

The saved preset file will contain structured data that enables accurate restoration of workspace layouts, including all necessary metadata for future compatibility and validation.

3 Phase 3: Development for Layout Restoration

This phase will focus on implementing the layout restoration functionality of the SwayLayoutManager tool. By building with the parsing and saving capabilities developed in phase 2, this phase will implement the `load` command that reads saved presets and reconstructs workspace layouts by launching applications and positioning them according to the saved configuration.

3.1 Load Command Implementation

The `swaylayoutmgr load <name>` command is the complementary functionality to the save command, enabling users to restore previously saved workspace layouts. This command will be implemented with the following detailed specification:

3.1.1 Command Execution Flow

1. **Preset Loading:** Read the configuration file from `~/.config/swaylayoutmanager/presets/<name>.j` and parse the JSON structure to extract the saved layout information.
2. **Workspace Restoration:** Iterate over each workspace defined in the preset and for each application window:

- Switch to the target workspace using `swaymsg workspace number <N>`
- Check if the application is already running by querying `app_id`/`class`
- Launch the application using `swaymsg exec <command>` if not already running
- Wait for the window to appear (configurable timeout)
- Use sway's criteria selection `[app_id="..." title="..."]` to focus the specific window
- Apply positioning commands to move and resize the window according to saved coordinates
- Restore floating state if applicable (`swaymsg floating enable/disable`)
- Set the workspace layout mode (`splith`, `splitv`, `tabbed`, or `stacked`)

3. **Layout Reconstruction:** Apply the saved layout structure by recreating container hierarchies and positioning windows according to their saved `rect` coordinates and sibling order.
4. **Verification:** Query the current layout tree to verify successful restoration and log any discrepancies or applications that failed to launch.

3.1.2 Error Handling Requirements

- Check if preset file exists before attempting to load
- Validate JSON structure and presence of all required fields
- Handle missing applications gracefully by logging warnings and continuing with remaining applications
- Implement timeout handling for applications that fail to launch or don't create windows within expected time
- Provide fallback positioning strategies if exact layout reconstruction fails (to be determined)
- Handle version compatibility issues with presets created using different sway versions (to be determined)
- Provide detailed error messages and restoration reports for troubleshooting

3.1.3 Implementation Considerations

The load command will need to handle asynchronous application launches and implement proper synchronization mechanisms to ensure windows are fully initialized before applying layout commands.